

CS1010J Lecture #7

Searching & Sorting

Bring order to the world

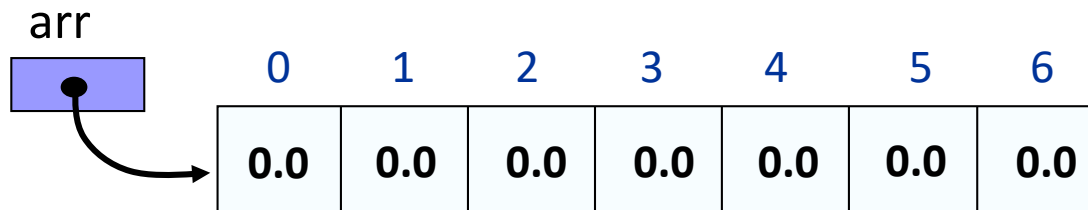


Department of Computer Science
School of Computing

Arrays

- **Array** is an indexed collection of homogeneous data.
 - Array index starts from 0.
- **Example:**
 - `arr[0]`, `arr[1]`, `arr[2]`...`arr[6]` are all **double** variables.
 - By default, they all have the value 0.0.

```
double[] arr = new double[7];
```



Creating Arrays

- **Method one:** use `new` operator to create an array with a specific size known at **compile time**.

```
int[] arr = new int[6];
```

- **Method Two:** use `new` operator to create an array with a specific size known at **runtime**.

```
int size = sc.nextInt();  
int[] arr = new int[size];
```

- **Method three:** create an array with known values using a pair of curly brackets.

```
int[] arr = {100, 300, 500};
```

Arrays as Parameters

- The reference to the array is passed into a method.
 - Array is shared between the caller and callee methods.

```
import java.util.*;
```

```
class Lec7Quiz1 {
```

```
    public static void main(String[] args) {  
        int[] list = { 22, 55, 33 };  
        swap(list, 0, 2);  
        System.out.println( Arrays.toString(list) );  
    }
```

[33, 55, 22]

Rename the array
list to **arr**

```
// To swap arr[i] with arr[j]
```

```
public static void swap(int[] arr, int i, int j) {  
    int temp = arr[i];  
    arr[i] = arr[j];  
    arr[j] = temp;  
}  
}
```

Arrays as Return Values

- Arrays can be returned from methods.

```
import java.util.*;
```

```
class Lec7Quiz2 {
```

```
    public static void main(String[] args) {  
        int[] values = makeArray(5, 60);  
        System.out.println( Arrays.toString(values) );  
    }
```

[60, 30, 20, 15, 12]

Receive an array
from method call

```
// To create an array and return it to caller
```

```
public static int[] makeArray(int size, int limit) {  
    int[] arr = new int[size];  
    for (int i = 0; i < arr.length; i++) {  
        arr[i] = limit/(i+1);  
    }  
    return arr;  
}
```

arr[0] <- 60/1

arr[1] <- 60/2

...

Quiz

(CS1010 AY2011/12 Semester 1 Midterm Test, Q13)

- What is the output of the following code?

```
import java.util.Arrays;

class Lec7Quiz3 {

    public static void main(String[] args) {

        int[] list = new int[5];

        for (int i = 0; i < 5; i++) {
            list[list[i]]++;
        }

        System.out.println( Arrays.toString(list) );
    }
}
```

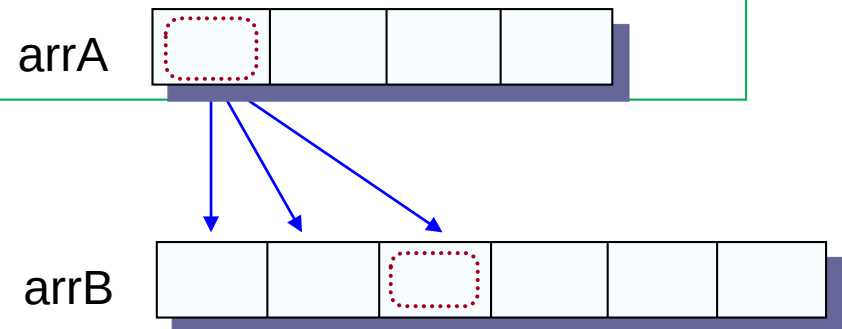
[5, 0, 0, 0, 0]

PS 3 Ex #14 Set Containment

```
// Return true if arrA is a subset of arrB, or false otherwise
public static boolean isSubset(int[] arrA, int[] arrB) {
    int count = 0;
    for (int i = 0; i < arrA.length; i++) {
        for (int j = 0; j < arrB.length; j++) {
            if (arrA[i] == arrB[j]) {
                count++;
                break;
            }
        }
    }

    // if all arrA elements appear in arrB,
    // count should be equal to arrA.length
    return arrA.length == count;
}
```

Version 1



PS 3 Ex #14 Set Containment

```
// Return true if arrA is a subset of arrB, or false otherwise
public static boolean isSubset(int[] arrA, int[] arrB) {

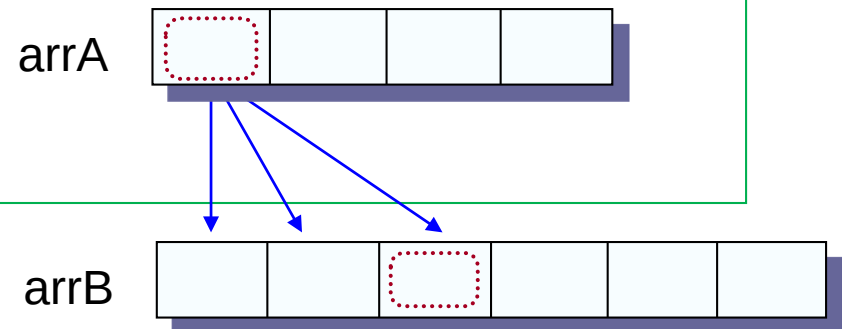
    boolean contains = true;

    for (int i = 0; i < arrA.length && contains; i++) {

        // assume arrA[i] is not in arrB, verify it
        contains = false;
        for (int j = 0; j < arrB.length && !contains; j++) {
            if (arrA[i] == arrB[j]) {
                contains = true;
            }
        }
    }

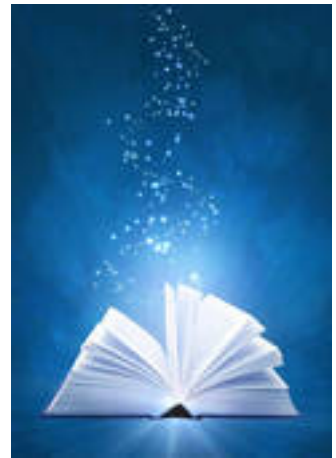
    return contains;
}
```

Version 2



Learning Objectives

- At the end of this lecture, you should understand:
 - The basic **searching** and **sorting** algorithms.
 - How to implement searching and sorting algorithms using arrays.



Introduction

- Today, we will study some simple yet useful **classical algorithms** which find their place in many CS applications.
 - **Searching** for some data amid very large collection of data.
 - **Sorting** large collection of data according to some order.
- We will begin with an algorithm (**idea**), then show how the algorithm is transformed into a Java program (**implementation**).
 - This brings back (reminds you) our very first lecture: the importance of beginning with an algorithm

Searching



- Searching is a common task that we carry out daily.
 - ❑ Searching for a location in a map.
 - ❑ Searching online for Java API specification.
 - ❑ Searching for a perfect girlfriend.
 - ❑ etc.
- In this lecture, you will learn how to search for an element (often called a **search key**) in an array.
 - ❑ Problem statement:

Given an array of data and a search key X , return the position of X in the array if it exists.

Linear Search : Algorithm



- Also known as Sequential Search.
- **Idea**: search the array from one end to the other end in linear progression.
- Example:

Search for key 24 in this array:

87	12	51	9	24	63
↑	↑	↑	↑	↑	
<i>no</i>	<i>no</i>	<i>no</i>	<i>no</i>	<i>yes!</i>	

Q: What to report if key is not found?

Linear Search : Program



```
// Search for key in arr using linear search.  
// Return index if found; or -1 otherwise.  
public static int linearSearch(int[] arr, int key) {  
    for (int i = 0; i < arr.length; i++) {  
        if (key == arr[i]) {  
            return i;  
        }  
    }  
  
    return -1; // not found  
}
```

Linear Search : Performance Analysis

- We use the **number of comparisons** algorithms make to analyze their performance.
 - The ideal searching algorithm will make the least possible number of comparisons to locate the desired data.
 - This topic is called **analysis of algorithms**, which will be formally introduced in CS1020.
- For an array with n elements:

Q: What is the maximum possible number of comparisons made?

n comparisons

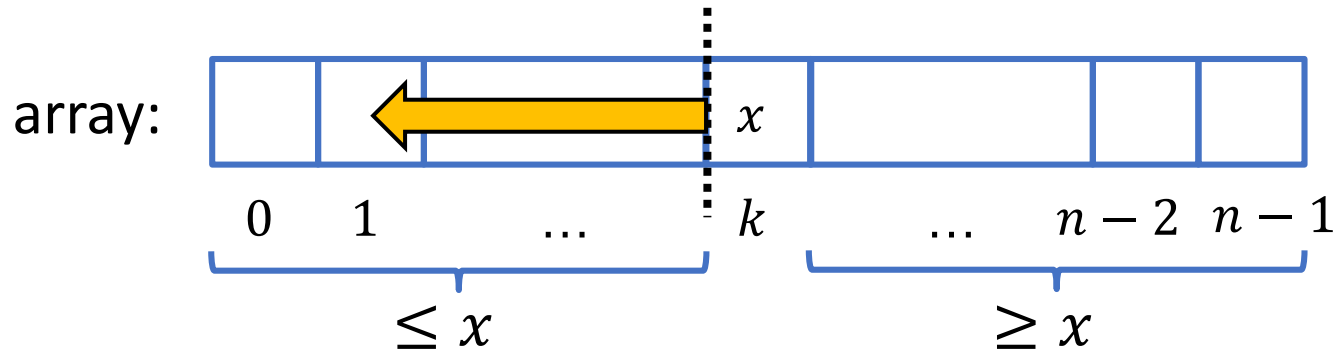
Q: Under what circumstances do we encounter such a worst case?

- (a) Key not found
- (b) Found at last position

Can we do better?
Yes, if the array is
sorted.

Binary Search

- Assuming an array is sorted in ascending order:



- if the k^{th} element (x) is larger than the **search key**, then we only need to search in the indices $< k$.

Sequence of Successful Search (1/3)

	low	high	mid
#1	0	8	4

search (**89**)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

0	1	2	3	4	5	6	7	8
7	13	20	38	44	52	88	89	92
↑ low		↑ mid			↑ high			

44 < **89** → low = mid+1 = 5

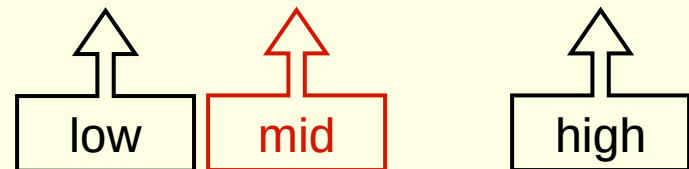
Sequence of Successful Search (2/3)

	low	high	mid
#1	0	8	4
#2	5	8	6

search (**89**)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

0	1	2	3	4	5	6	7	8
					52	88	89	92



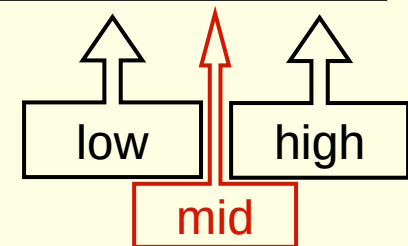
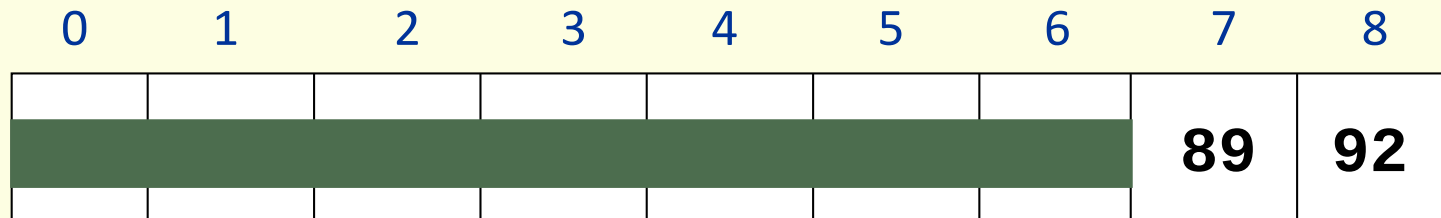
88 < **89** → low = mid+1 = 7

Sequence of Successful Search (3/3)

	low	high	mid
#1	0	8	4
#2	5	8	6
#3	7	8	7

search (**89**)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



89 = 89 → **Successful search**

Sequence of Unsuccessful Search (1/4)

	low	high	mid
#1	0	8	4

search (**50**)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

0	1	2	3	4	5	6	7	8
7	13	20	38	44	52	88	89	92
↑ low				↑ mid				↑ high

$44 < \mathbf{50} \longrightarrow low = mid + 1 = 5$

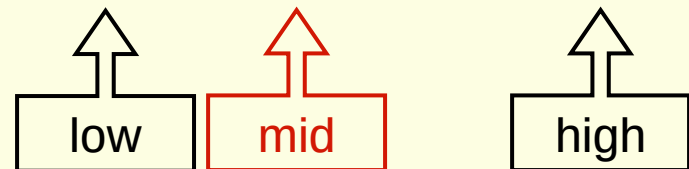
Sequence of Unsuccessful Search (2/4)

	low	high	mid
#1	0	8	4
#2	5	8	6

search (**50**)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$

0	1	2	3	4	5	6	7	8
					52	88	89	92



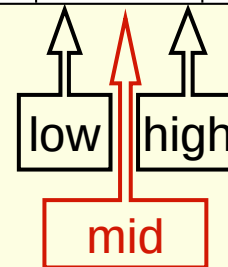
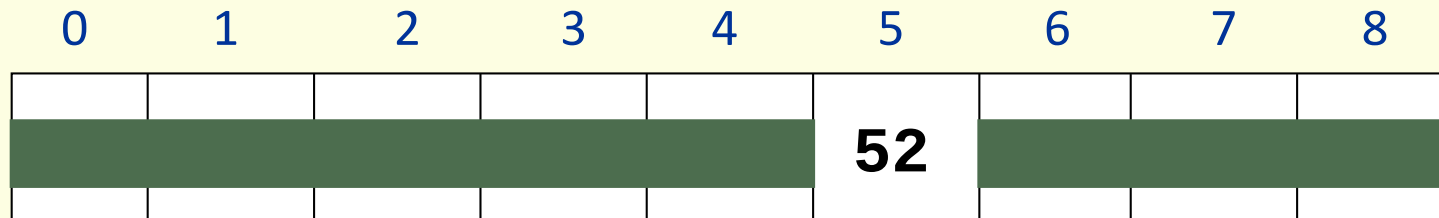
88 > **50** → **high = mid-1 = 5**

Sequence of Unsuccessful Search (3/4)

	low	high	mid
#1	0	8	4
#2	5	8	6
#3	5	5	5

search (**50**)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



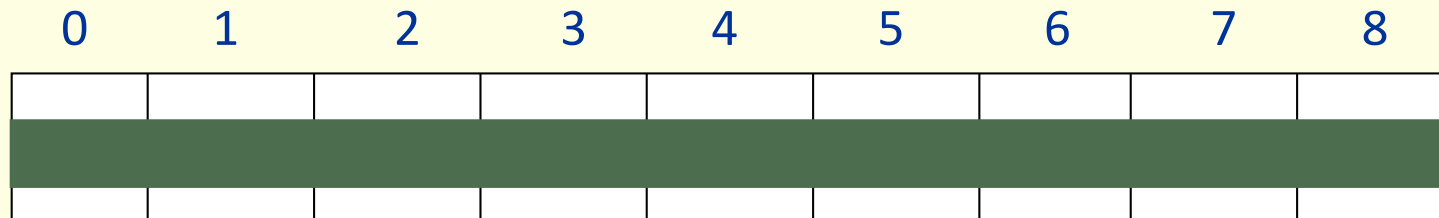
52 > **50** → **high = mid-1 = 4**

Sequence of Unsuccessful Search (4/4)

	low	high	mid
#1	0	8	4
#2	5	8	6
#3	5	5	5
#4	5	4	

search (**50**)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



Unsuccessful Search

low > high no more to search

Binary Search : Algorithm

(Pre-condition: array is sorted in ascending order)

■ Algorithm

- Look for the **key** in the middle position of the array.

Either of the following 2 cases happens:

- If the **key** is **smaller** than the middle element, then “discard” the right half of the array and repeat the process on the left half of the array.
- If the **key** is **greater** than the middle element, then “discard” the left half of the array and repeat the process.
- Terminating condition: either the **key** is found, or when all elements have been “discarded”.

Binary Search : Program

```
// Search for key in sorted 'arr' using binary search
// Return index if found; return -1 otherwise
public static int binarySearch(int[] arr, int key) {

    int low = 0, high = arr.length-1;;

    while (low <= high) {
        int mid = (low+high) / 2;
        if (key == arr[mid]) {
            return mid;
        } else if (key < arr[mid]) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    } // end while

    return -1;
}
```

Binary Search : Analysis

- In binary search, each step eliminates the problem size (scope of search) by half.
 - The problem size gets reduced to 1 very quickly (see next slides).
- This is a simple yet powerful strategy, of halving the solution space in each step.
- Such a strategy, a special case of divide-and-conquer paradigm, can be naturally implemented using recursion (a topic to be covered later).

Binary Search : Performance Analysis

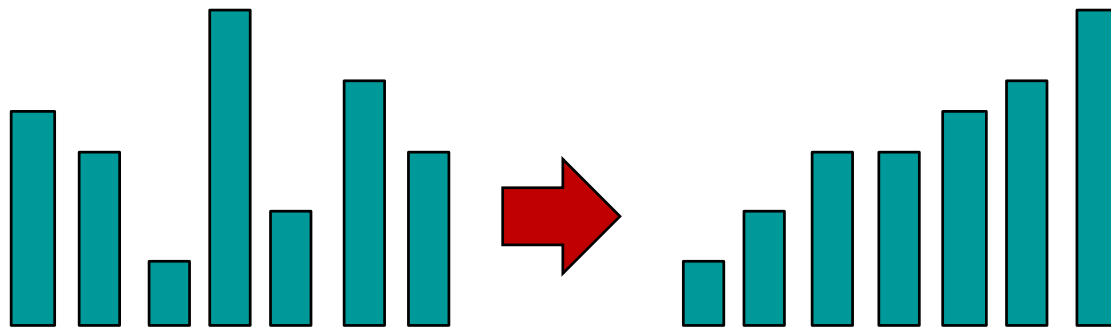
■ Worst-case analysis:

Array size n	Linear Search (n comparisons)	Binary Search ($\log_2 n$ comparisons)
100	100	≈ 7
1,000	1,000	≈ 10
10,000	10,000	≈ 14
100,000	100,000	≈ 17
1,000,000	1,000,000	≈ 20
10^9	10^9	≈ 30

To apply binary
search,
The array must be
sorted.

Sorting

- Sorting is an process of arranging items in sequence.
 - once a sequence of items is sorted, many problems (such as searching) become easy.



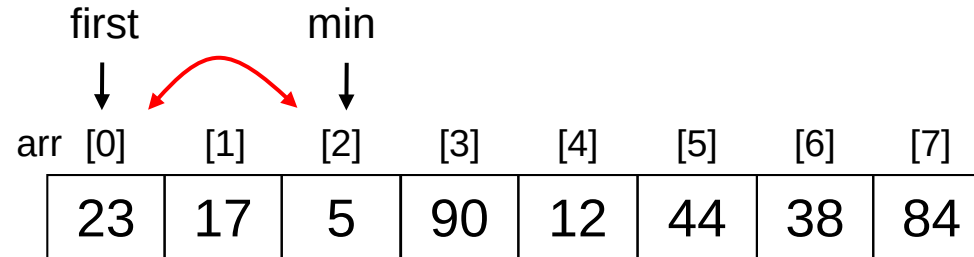
- Problem statement:

Given an array of data, arrange all data in ascending order.

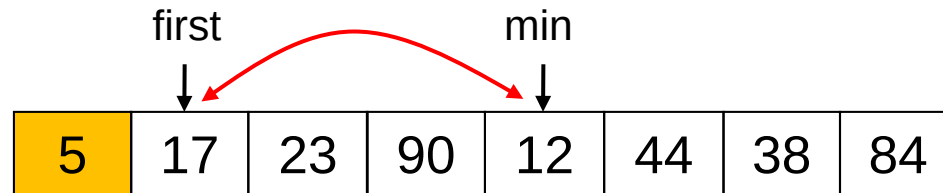
Selection Sort : Illustration (1/2)

$n = 8$

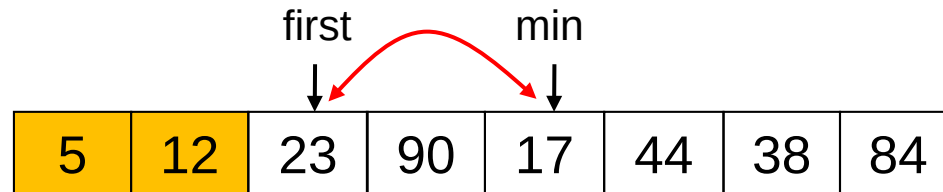
1st pass:



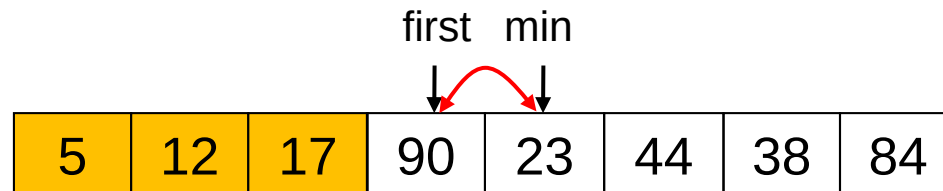
2nd pass:



3rd pass:



4th pass:



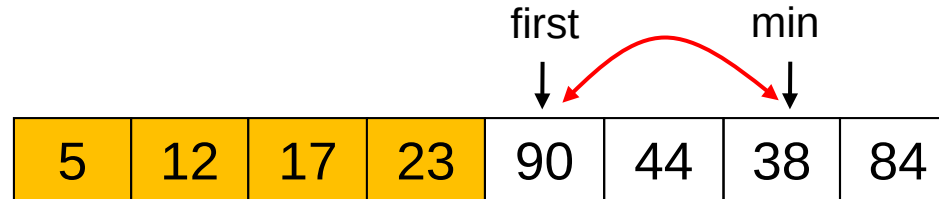
Selection Sort : Illustration (2/2)

Q: How many passes are needed for an array of n elements?

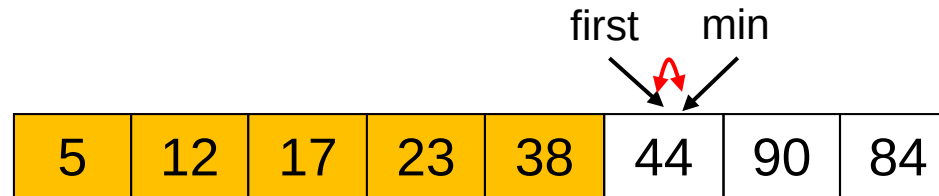
$n-1$

$n = 8$

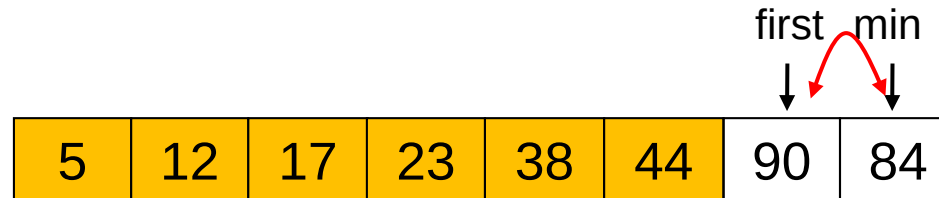
5th pass:



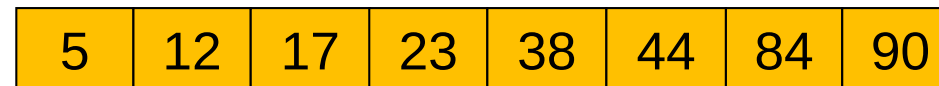
6th pass:



7th pass:



Final array:



Selection Sort : Algorithm

■ Algorithm

- ❑ **Step 1:** Find the smallest element in the array.
- ❑ **Step 2:** Swap this smallest element with the element in the first position. (Now, the smallest element is in the right place.)
- ❑ **Step 3:** Repeat steps 1 and 2 with the array having one fewer element (i.e. the smallest element just found and placed is “exempted” from further processing).

Selection Sort : Implementation

```
// Sort array 'arr' in increasing order
public static void selectionSort(int[] arr) {

    // each iteration of the for loop is one pass
    for (int startIdx = 0; startIdx < arr.length-1; startIdx++) {
        // find the index of the minimum element
        int minIndex = startIdx;
        for (int i = startIdx+1; i < arr.length; i++) {
            if (arr[i] < arr[minIndex]) {
                minIndex = i;
            }
        }
        // swap minimum element with element at startIdx
        int temp = arr[startIdx];
        arr[startIdx] = arr[minIndex];
        arr[minIndex] = temp;
    } // end for loop
}
```

Selection Sort : Performance

**Non-
examinable**

- We choose **the number of comparisons** as our basis of analysis.
- Comparisons of array elements occur **in the inner loop**, where the minimum element is determined.
- Assuming an array with n elements. Table below shows the number of comparisons for each pass.
- The total number of comparisons is calculated in the formula below.
- Such an algorithm is call an **n^2 algorithm**, or **quadratic algorithm**, in terms of running time complexity.

Pass	#comparisons
1	$n - 1$
2	$n - 2$
3	$n - 3$
...	...
$n - 1$	1

$$\sum_{i=1}^{n-1} i = \frac{(n-1)(n)}{2} = \frac{n^2 - n}{2} \cong n^2$$

Bubble Sort

- Bubble sort is another classic sorting algorithm. The key idea is to make **pairwise comparisons** and exchange the positions of the pair if they are out of order.
- A YouTube video clip to simulate bubble sort:
 - ❑ <http://www.youtube.com/watch?v=lyZQPjUT5B4>
- Animated sorting algorithms:
 - ❑ <http://www.sorting-algorithms.com/>
 - ❑ <http://www.cs.ubc.ca/~harrison/Java/sorting-demo.html>



One Pass of Bubble Sort

0	1	2	3	4	5	6	7	8
23	17	5	90	12	44	38	84	77

↑ exchange

17	23	5	90	12	44	38	84	77
----	----	---	----	----	----	----	----	----

↑ exchange

17	5	23	90	12	44	38	84	77
----	---	----	----	----	----	----	----	----

↑ ok
↑ exchange

17	5	23	12	90	44	38	84	77
----	---	----	----	----	----	----	----	----

↑ exchange

17	5	23	12	44	90	38	84	77
----	---	----	----	----	----	----	----	----

exchange ↑

17	5	23	12	44	38	90	84	77
----	---	----	----	----	----	----	----	----

exchange ↑

17	5	23	12	44	38	84	90	77
----	---	----	----	----	----	----	----	----

exchange ↑

17	5	23	12	44	38	84	77	90
----	---	----	----	----	----	----	----	----

Q: Is the array sorted?

Q: What did we achieve?

Bubble Sort : Program

```
// Sort array 'arr' in increasing order
public static void bubbleSort(int[] arr) {

    for (int end = arr.length-1; end > 0; end--) {

        for (int i = 0; i < end; i++) { // one pass
            if ( arr[i] > arr[i+1] ) { // out of order
                int temp = arr[i]; // swap
                arr[i] = arr[i+1];
                arr[i+1] = temp;
            }
        } // end inner for loop

    } // end outer for loop
}
```

Bubble Sort : Performance

**Non-
examinable**

- Bubble sort requires $n - 1$ passes to sort an array with n elements. Comparisons of array elements occur in the inner loop.
- The number of comparisons in each pass is given in the table below and the total number of comparisons is calculated in the formula below.
- Like Selection Sort, Bubble sort is also an n^2 algorithm, or quadratic algorithm, in terms of running time complexity.

Pass	#comparisons
1	$n - 1$
2	$n - 2$
3	$n - 3$
...	...
$n - 1$	1

$$\sum_{i=1}^{n-1} i = \frac{(n-1)(n)}{2} = \frac{n^2 - n}{2} \cong n^2$$

Bubble Sort : Enhanced Version

- It is possible to enhance bubble sort algorithm to reduce the number of passes.
 - Suppose that in a certain pass, no swap is needed. This implies that the array is already sorted, and hence the algorithm may terminate without going on to the next pass.
- We will discuss this enhanced version in the coming tutorial.

More Sorting Algorithms

- We have introduced 2 basic sorting algorithms **Bubble Sort** and **Selection Sort**. The third classic sorting algorithms is **Insertion Sort**.
 - ❑ These 3 are the simplest sorting algorithms.
 - ❑ However, they are very slow, as their running time complexity is quadratic.
- Faster sorting algorithms exist and are the topics of more advanced programming modules.
 - ❑ Merge Sort (CS2040)
 - ❑ Quick Sort (CS2040)
 - ❑ Heap Sort (CS2040)

Today's Summary

Searching and Sorting

Searching algorithms

- Linear (sequential) search
- Binary search

Sorting algorithms

- Selection sort
- Bubble sort

